Proyecto Final: Motor/Gestor de Bases de Datos con Árboles Auto balanceados.

Documentación técnica.

Juan Felipe Gallego Chaves – 20251020016

Juan Esteban Guerrero Nieto – 20251020053

Roger Steward Arenales Sanabria - 20251020056

Ciencias de la Computación I

Simar Enrique Herrera Jiménez

Grupo 020-81

Facultad Ingeniería

Universidad Distrital Francisco José de Caldas

29/05/2026

Especificación del lenguaje.

- **Lenguaje usado para la producción de la app:** Java.
- **JDK:** Java 16 o superior.
- **Distribución:** Oracle Java.
- **Entorno de ejecución/Software de desarrollo:** Eclipse IDE for Enterprise Java and Web Developers - 2026-03

Comandos soportados: En la consola, para el control de la aplicación se permite:

Comando	Descripción
CREATE	Creará una nueva tabla, seguido por un espacio y un nombre para la misma. Simplemente (dejando espacios entre valores): "CREATE NOMBRE_DE_TABLA"
SHOW	Mostrará simplemente la lista de tablas existentes.
INSERT	Usado para agregar nuevos datos a una tabla existente. De la siguiente manera (dejando espacios entre valores): "INSERT NOMBRE_DE_TABLA Datos" Para especificar valores, usar el signo (=) de esta manera: nombre=José.
FIND	Se usa en el momento de buscar información en las tablas. De la siguiente manera (dejando espacios entre valores): "FIND NOMBRE_DE_TABLA valór_asignado"
UPDATE	Usado para modificar datos en una tabla existente. De la siguiente manera (dejando espacios entre valores): "UPDATE NOMBRE_DE_TABLA valór_asignado Datos=Nuevo_Dato"
DELETE	Se usa para eliminar datos de una tabla. De la siguiente manera (dejando espacios entre valores): "DELETE NOMBRE_DE_TABLA valór_asignado"
INORDER	Se usa para darle un vistazo a los datos de una tabla en específico. Simplemente (dejando espacios entre valores): "INORDER NOMBRE_DE_TABLA"
RANGE	Busca en un rango de valores los datos cuyo valor asignado se encuentre dentro de ese intervalo. Simplemente (dejando espacios entre valores): "RANGE NOMBRE_DE_TABLA valor_inicial valor_final"
DROP	Eliminar una tabla. Simplemente (dejando espacios entre valores): "DROP NOMBRE_DE_TABLA"
EXIT	Cierra sesión.
LOAD DATASETS	Carga un data set ya existente.

Diseño arquitectónico por módulos.

- **Módulo Parser:**

El módulo Parser interpreta comandos de consola y transforma datos entre formatos. En avl.ConsolaDB, el método ejecutarComando(String) parsea entrada separando claves (CREATE/INSERT/FIND/DELETE/UPDATE/DROP/SHOW/INORDER/RANGE) de argumentos mediante split. Para INSERT, extrae pares campo=valor; para RANGE, convierte parámetros a enteros.

En persistencia.JsonManager, cargarRegistros(AVL) parsea los archivos línea por línea separando clave de campos con delimitador semicolon, luego extrae pares campo-valor. En modelo.Registro, toString() formatea el HashMap interno (estructura de datos por medio de indices) como texto legible.

- **Módulo de Persistencia/Almacenamiento:**

El módulo usa archivos de texto plano con delimitadores semicolón. Cada tabla genera “Nombre_Tabla.json” con formato clave;campo1=valor1;campo2=valor2; por línea. El archivo tablas.txt almacena metadatos con formato nombreTabla;campo1 por línea.

En gestor.DatabaseManager se gestiona múltiples tablas mediante HashMaps de tablas (nombre → Tabla con AVL) y archivos (nombre → JsonManager). El método crearTabla(nombre, campos) instancia Tabla, carga datos previos e inicializa la persistencia. Metodo eliminarTabla(nombre) remueve referencias y elimina físicamente el archivo JSON. Inserción es append-only, mientras actualización/eliminación requiere reescritura completa del archivo. La complejidad de carga es O(n log n) para n registros.

- **Módulo de Vista:**

El módulo UI (interfaz de usuario) implementa dos interfaces: consola (ConsolaDB) y GUI (VentanaDB). ConsolaDB ejecuta un bucle continuo con “prompt”, lee comandos y valida con try-catch. “MainConsola” es el punto de entrada para “consola”.

VentanaDB crea una ventana principal con panel de botones: SELECCIONAR TABLA, GESTIONAR ÁRBOLES, GESTIONAR REGISTROS, MOSTRAR ÁRBOL. La clase “panelArbol” dibuja el árbol AVL recursivamente con nodos circulares, aristas, altura

y factor de balance visibles. El método calcularPosiciones() asigna coordenadas reduciendo el gap horizontal por nivel. Un Timer resalta nodos temporalmente.

- **Módulo Índice:**

El módulo índice gestiona búsquedas eficientes ordenadas por clave numérica, implementado en avl.AVL y público por modelo.Tabla.getArbol(). La API pública incluye insertar(), buscar(), actualizar(), eliminar(), buscarRango(). Todo en ArrayList, inorderTexto() y getRaiz(). Cada tabla es un árbol AVL independiente en HashMap.

"Tabla" contiene nombre, lista de campos (esquema dinámico) y AVL. El método agregarCampo() añade al esquema y actualiza todos los registros; eliminarCampo() remueve y sincroniza el árbol.

- **Módulo API:** El módulo API define interfaces públicas para consumo entre módulos.

API de índice en AVL.java:

insertar(), buscar(), actualizar(), eliminar(), buscarRango(), inorderTexto(), getRaiz().

API de almacenamiento en DatabaseManager.java:

crearTabla(), eliminarTabla(), obtenerTabla(), obtenerArchivo(), existeTabla(), obtenerNombresTablas(), agregarCampoTabla(), eliminarCampoTabla().

API de persistencia en JsonManager.java: guardarRegistro(), limpiarArchivo(), cargarRegistros(). API de modelo en Registro.java: agregarCampo(), obtenerCampo(), getDatos(), eliminarCampo().

API de Tabla.java:

getNombre(), getCampos(), getArbol(), agregarCampo(), eliminarCampo().

Diseño del árbol.

El sistema implementa un Árbol Binario de Búsqueda Balanceado AVL como estructura de índice para cada tabla creada que satisface la propiedad de balanceo. En la implementación, cada objeto "Nodo" almacena explícitamente su altura, y el método usado para balancear calcula el factor de balance en una complejidad O(1):

Propiedades Matemáticas:

Propiedad	Descripción
Orden binario	Para todo nodo: clave_izq ≤ clave_nodo < clave_der
Factor de Balance	BF(n) = altura(der) - altura(izq)
Invariante AVL	$\lvert BF(n) \rvert \leq 1$ para todos los nodos
Altura máxima	$h \leq 1.44 \cdot \log_2(n + 2)$ (garantía logarítmica)

Algoritmo de Balanceo:

Caso	Condición	Tipo	Rotación
Der-Der	BF > 1 y clave > nodo.der.clave	Simple	rotarIzq()
Izq-Izq	BF < -1 y clave ≤ nodo.izq.clave	Simple	rotarDer()
Der-Izq	BF > 1 y clave ≤ nodo.der.clave	Doble	rotarDer(der) + rotarIzq()
Izq-Der	BF < -1 y clave > nodo.izq.clave	Doble	rotarIzq(izq) + rotarDer

Rotaciones: Reestructuran localmente el árbol en O(1), preservando el recorrido in-order (orden de claves). Actualizan alturas de nodos afectados inmediatamente.

Visualización: Cargar: Parseo manual de JSON línea por línea, inserta en AVL.

Formato de persistencia.

El sistema usa archivos JSON en texto plano como mecanismo de persistencia. Cada tabla tiene su propio archivo JSON independiente: "(Nombre_Tabla).json." La inserción es append-only (abre/cierra archivo en cada registro), mientras que actualizar/eliminar hace reescritura completa (borra todo y reescribe el árbol AVL completo en orden "in-order"). Posee recuperación básica, se aplica al reiniciar, recuperando solo lo que se había guardado exitosamente antes de cerrar.

Análisis de Complejidad.

Se garantiza O(log n) para búsqueda, inserción y eliminación en el árbol AVL, siempre y cuando el árbol permanezca balanceado correctamente. El presente proyecto hace uso de técnicas en código que implementan correctamente el algoritmo AVL clásico con rotaciones que mantienen el invariante de balance.

Sin embargo, el proyecto hace uso de distintas operaciones que demuestran complejidades divergentes:

- **O(log n + k):** Donde k = número de nodos en el rango, esto se presenta en el momento que busca el rango de valores asignados. Garantizando la complejidad O(log n) más los pasos de búsqueda. Pero k puede ser n en cualquier momento.
- **O(n):** Al realizar la inserción de nuevos nodos/datos en una tabla, debe recorrer todo el árbol.

Tabla de análisis:

Operación	Método	Complejidad	¿Garantiza O(log n)?	Nota
Búsqueda	buscar()	O(log n)	Sí	Peor caso garantizado por AVL
Inserción	insertar()	O(log n)	Sí	Máximo 2 rotaciones
Eliminación	eliminar()	O(log n)	Sí	Rebalanceo puede propagarse
Actualización	actualizar()	O(log n)	Sí	Solo busca, sin recolocar
Búsqueda por Rango	buscarRango()	O(log n + k)	Varía	k puede ser hasta n
Guardar Archivo	guardarEnArchivo()	O(n)	No	Recorre todo el árbol
Cargar Personas	cargarPersonas()	O(n)	No	Parsea cada línea del JSON